ring oscillator

Project 1 — CMOS Ring Oscillator

IC Design with Open-Source EDA Tools *Simulation and PVT Analysis using ngspice + Sky130 PDK*

Author	Toney
Level	BTech / Beginner
Tools	ngspice 36, Sky130 PDK, WSL2 / Ubuntu 22.04
Process	SkyWater Sky130 (130 nm CMOS)
Supply Voltage	1.8 V
Date	May 2026
Series	Part 1 of 8 — Summer IC Design Series

Table of Contents

---

1. Project Overview

1.1 Objective

This report documents the design, simulation, and analysis of a 3-stage CMOS ring oscillator using entirely open-source EDA tools on a Windows 11 machine via WSL2/Ubuntu. The project covers:

1. Setting up a professional Linux EDA environment on Windows using WSL2.
2. Writing a SPICE netlist for a CMOS ring oscillator using Sky130 transistor models.
3. Running transient simulation and measuring oscillation frequency with `ngspice`.
4. Performing PVT (Process, Voltage, Temperature) analysis across worst-case corners.

1.2 Tools and Environment

Tool	Version	Purpose
WSL2 + Ubuntu	22.04 LTS	Linux environment on Windows 11
ngspice	36	SPICE circuit simulator
Python	3.10.12	Required for volare PDK manager
volare	0.20.6	Sky130 PDK version managers
Sky130 PDK	bdc9412b	130 nm open-source process design kit
nano	6.2	Terminal text editor for netlists

1.3 Project Outcome

> **Key result:** The ring oscillator oscillates at **1.77 GHz** at the typical process corner (TT), 27°C, and 1.8 V supply. Under worst-case PVT conditions (SS corner, 100°C, 1.62 V), frequency drops to **967 MHz** — a 45% reduction, directly demonstrating why Static Timing Analysis (STA) is essential in production silicon.

---

2. Environment Setup

2.1 Why WSL2?

All professional EDA tools (ngspice, Magic, KLayout, OpenROAD) are built for Linux. Since the development machine runs Windows 11, WSL2 (Windows Subsystem for Linux 2) was used. WSL2 runs a real Linux kernel alongside Windows without the overhead of a full virtual machine.

Property	WSL2	Virtual Machine
Startup time	~2 seconds	30–60 seconds
RAM usage	Shared, dynamic	Fixed, locked upfront
Disk space	~1–2 GB	20–40 GB
File access	Seamless both ways	Needs shared folder setup
Best for	Development / EDA	Full OS isolation

2.2 Prerequisites

The following were confirmed before installation:

- Windows 11 — verified via `winver`
- Virtualisation enabled — confirmed via Task Manager → Performance → CPU

2.3 Installation Steps

2.3.1 WSL2 and Ubuntu

Opened PowerShell as Administrator and ran:

```
wsl --install
# After restart, Ubuntu did not auto-launch, so installed manually:
wsl --install -d Ubuntu-22.04
```

Ubuntu opened directly in the terminal with the prompt: `toney@Toney:/mnt/c/WINDOWS/system32$`

Moved to home directory:

```
cd ~
pwd   # /home/toney
```

2.3.2 System Update

```
sudo apt update && sudo apt upgrade -y
```

2.3.3 ngspice

```
sudo apt install -y ngspice
ngspice --version   # ngspice-36
```

2.3.4 Python and pip

```
sudo apt install -y python3 python3-pip
python3 --version   # Python 3.10.12
```

2.3.5 volare and Sky130 PDK

```
pip3 install volare

# Fix PATH so volare command is found:
echo 'export PATH=$PATH:$HOME/.local/bin' >> ~/.bashrc
source ~/.bashrc

# Download Sky130 PDK (explicit commit hash required outside OpenLane):
export PDK_ROOT=$HOME/pdk
volare enable --pdk sky130 --pdk-root $PDK_ROOT \
    bdc9412b3e468c102d01b7cf6337be06ec6e9c9a

# Make PDK_ROOT permanent:
echo 'export PDK_ROOT=$HOME/pdk' >> ~/.bashrc
source ~/.bashrc

# Verify:
ls $PDK_ROOT/sky130A/libs.tech/ngspice/
```

The PDK directory contained `.spice`, `.lib.spice`, and `.model.spice` files — confirming successful installation.

> **Debugging note:** The volare command initially failed with *"could not determine open_pdk version: tool_metadata.yml not found"*. This occurs when volare is run outside an OpenLane project directory, which normally contains that file. The fix is to supply the commit hash explicitly with the `bdc9412b...` argument shown above.

2.3.6 Project Directory

```
mkdir -p ~/ic_projects/project1_ring_oscillator
cd ~/ic_projects/project1_ring_oscillator
pwd   # /home/toney/ic_projects/project1_ring_oscillator
```

---

3. Theory

3.1 What is a Ring Oscillator?

A ring oscillator generates a periodic clock signal with no external input by connecting an **odd number of inverters in a loop**. Each inverter flips the signal (HIGH → LOW, LOW → HIGH) and introduces a propagation delay `t_pd`. Because there is an odd number of inversions, the loop can never settle to a stable state, so it oscillates continuously.

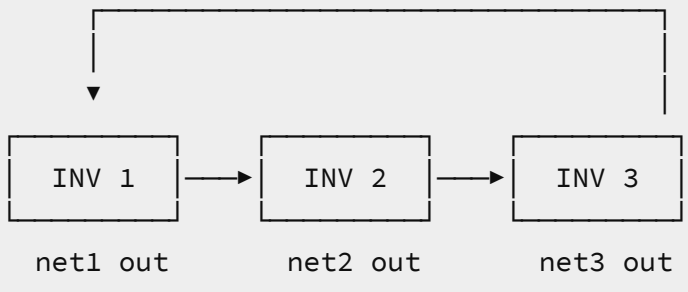


Figure: 3-stage ring oscillator loop. The output of INV 3 feeds back to the input of INV 1.

Oscillation Frequency

```
f = 1 / (2 × N × t_pd)
```

Where `N` is the number of inverter stages (3 here) and `t_pd` is the propagation delay of a single inverter. The factor of 2 accounts for one full HIGH→LOW→HIGH cycle.

Why an Odd Number of Stages?

With an even number of inverters the loop finds a stable logic state and stops switching — no oscillation. With an odd number, every possible state creates a logical contradiction, forcing the loop to keep toggling forever.

3.2 CMOS Inverter

Each stage is a CMOS inverter: a PMOS transistor pulls the output to VDD when the input is LOW, while an NMOS transistor pulls the output to GND when the input is HIGH. They form a complementary switch — when one is ON the other is OFF.

Transistor	Type	ON when input is	Output driven to
M_p (PMOS)	P-channel	LOW (0 V)	VDD = 1.8 V (HIGH)
M_n (NMOS)	N-channel	HIGH (1.8 V)	GND = 0 V (LOW)

3.3 Sky130 PDK

The SkyWater Sky130 PDK is a fully open-source 130 nm CMOS process design kit released by Google and SkyWater Technology in 2020. Key facts:

- Minimum transistor gate length: `L_min = 0.15 µm`
- Nominal supply voltage: 1.8 V
- Transistors are defined as *subcircuits* containing internal parasitics. They must be instantiated with the `X` prefix in SPICE,

not `M`.
- Model files are organised by process corner (`tt`, `ss`, `ff`, `sf`, `fs`).

---

4. Netlist Design

4.1 SPICE Netlist Format

In ngspice, a circuit is described in a plain-text netlist file (`.sp` extension). Each line declares one component and the nodes (wires) it connects to. The general format for a Sky130 MOSFET instance is:

```
Xname  drain  gate  source  bulk  ModelName  W=<width>  L=<length>
```

The file was created and edited using the `nano` terminal editor:

```
nano ring_oscillator.sp
```

4.2 Final Netlist

```
* Ring Oscillator - 3 stage
* Sky130 CMOS Process

* Load the Sky130 PDK tt (typical-typical) corner
.lib /home/toney/pdk/sky130A/libs.tech/ngspice/sky130.lib.spice tt

* Supply voltage
VDD vdd 0 1.8

* Inverter 1 - PMOS + NMOS (X prefix: Sky130 models are subcircuits)
X1_p net1 net3 vdd vdd sky130_fd_pr__pfet_01v8 W=1 L=0.15
X1_n net1 net3 0   0   sky130_fd_pr__nfet_01v8 W=0.5 L=0.15

* Inverter 2
X2_p net2 net1 vdd vdd sky130_fd_pr__pfet_01v8 W=1 L=0.15
X2_n net2 net1 0   0   sky130_fd_pr__nfet_01v8 W=0.5 L=0.15

* Inverter 3
```

```
X3_p net3 net2 vdd vdd sky130_fd_pr__pfet_01v8 W=1 L=0.15
X3_n net3 net2 0   0   sky130_fd_pr__nfet_01v8 W=0.5 L=0.15

* 10 fF capacitors to model parasitic capacitance and aid startup
C1 net1 0 10f
C2 net2 0 10f
C3 net3 0 10f

* Initial conditions: force unequal states to kick-start oscillation
.ic V(net1)=1.8 V(net2)=0 V(net3)=1.8

* Transient simulation: step=1 ps, stop=10 ns
.tran 1p 10n

.control
run
plot V(net1) V(net2) V(net3)
.endc

.end
```

4.3 Netlist Line-by-Line Explanation

Line / Element	Meaning
`.lib ... tt`	Load Sky130 transistor models for the Typical-Typical corner
`VDD vdd 0 1.8`	1.8 V DC voltage source between node `vdd` and ground
`X1_p net1 net3 vdd vdd ...`	PMOS: drain=net1, gate=net3, source=vdd, bulk=vdd
`X1_n net1 net3 0 0 ...`	NMOS: drain=net1, gate=net3, source=GND, bulk=GND
`W=1 L=0.15`	Width = 1 µm, Length = 0.15 µm (minimum for Sky130)
`C1 net1 0 10f`	10 fF parasitic capacitance on node net1 to ground
`.ic V(net1)=1.8`	Force net1 to start HIGH. Without this, the simulator may not break symmetry and oscillation will not start
`.tran 1p 10n`	Transient analysis: timestep = 1 ps, total runtime = 10 ns

> **Critical SPICE rule — `X` not `M`:** Sky130 transistors are wrapped inside subcircuit definitions to include gate resistance, junction capacitance, and other parasitics. SPICE requires subcircuit instances to start with `X`. Using `M` (the primitive MOSFET prefix)

> causes ngspice to fail with *"can't find model sky130_fd_pr__pfet_01v8"* because it searches the primitive model table, not the subcircuit library.

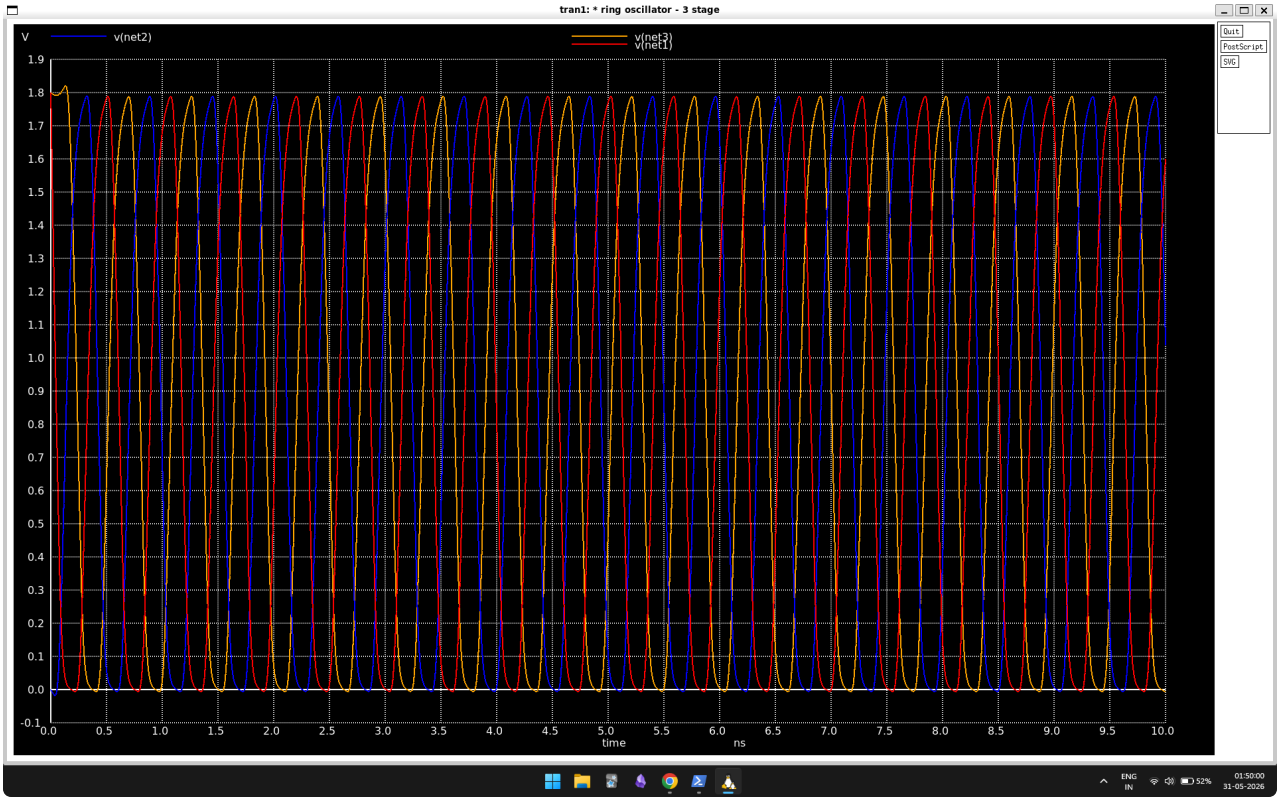
---

5. Simulation 1: Transient Waveform

5.1 Running the Simulation

```
ngspice ring_oscillator.sp
```

5.2 Terminal Output and graph




```
Circuit: * ring oscillator - 3 stage

Doing analysis at TEMP = 27.000000 and TNOM = 27.000000

Initial Transient Solution
--------------------------
Node                                   Voltage
----                                   -------
```

```
vdd                                    1.8
net1                                   1.8
net3                                   1.8
net2                                     0
vdd#branch                      -3.19918e-10

Reference value :  0.00000e+00
No. of Data Rows : 10008
```

5.3 Interpretation

The initial conditions show net1 and net3 at 1.8 V and net2 at 0 V — exactly as set by the `.ic` statement. The `vdd#branch` current of `-3.2 × 10⁻¹⁰ A` (0.32 nA) is the leakage current at room temperature, essentially zero for a healthy digital circuit at 27°C.

A waveform window opens showing three alternating square waves on net1, net2, and net3, each phase-shifted by one inverter delay — confirming the ring oscillator is working correctly.

---

6. Simulation 2: Frequency Measurement

6.1 Updated Control Block

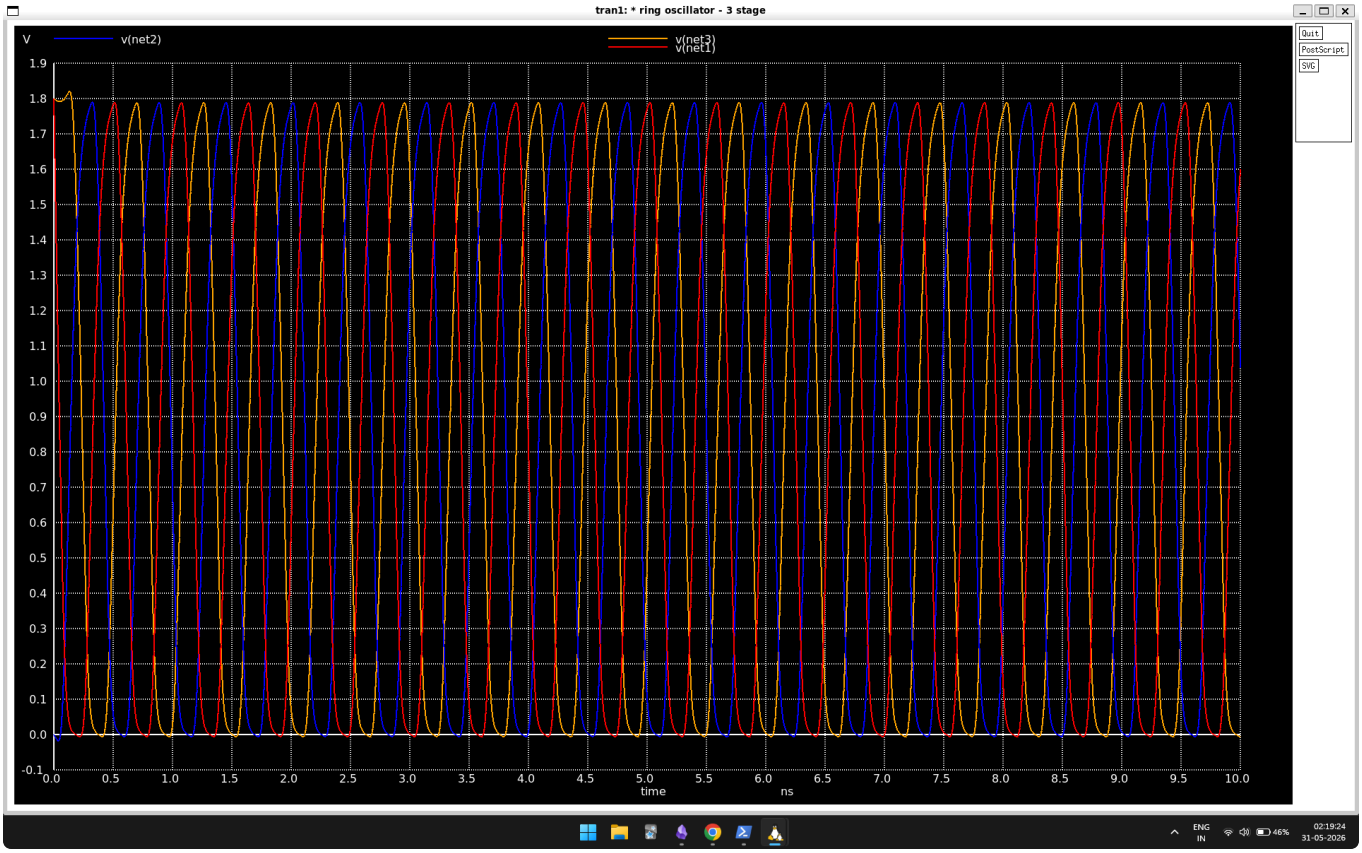
The `.control` block was updated to add a `meas` command that calculates the period precisely, then derives the frequency:

```
* Ring Oscillator - 3 stage
* Sky130 CMOS Process

* Load the Sky130 PDK tt (typical-typical) corner
.lib /home/toney/pdk/sky130A/libs.tech/ngspice/sky130.lib.spice tt

* Supply voltage
VDD vdd 0 1.8

* Inverter 1 - PMOS + NMOS (X prefix: Sky130 models are subcircuits)
X1_p net1 net3 vdd vdd sky130_fd_pr__pfet_01v8 W=1 L=0.15
X1_n net1 net3 0   0   sky130_fd_pr__nfet_01v8 W=0.5 L=0.15
```

```
* Inverter 2
X2_p net2 net1 vdd vdd sky130_fd_pr__pfet_01v8 W=1 L=0.15
X2_n net2 net1 0   0   sky130_fd_pr__nfet_01v8 W=0.5 L=0.15

* Inverter 3
X3_p net3 net2 vdd vdd sky130_fd_pr__pfet_01v8 W=1 L=0.15
X3_n net3 net2 0   0   sky130_fd_pr__nfet_01v8 W=0.5 L=0.15

* 10 fF capacitors to model parasitic capacitance and aid startup
C1 net1 0 10f
C2 net2 0 10f
C3 net3 0 10f

* Initial conditions: force unequal states to kick-start oscillation
.ic V(net1)=1.8 V(net2)=0 V(net3)=1.8

* Transient simulation: step=1 ps, stop=10 ns
.tran 1p 10n

* here is the main change when compared to the above netlist

.control
run
plot V(net1) V(net2) V(net3)

* Measure period: time between 1st and 2nd rising edge at V=0.9 (half VDD)
meas tran period_val TRIG v(net1) VAL=0.9 RISE=1 TARG v(net1) VAL=0.9 RISE=2

* Frequency = 1 / period
let freq_val = 1 / period_val
print freq_val
.endc
```

The `meas` command records the time at which `V(net1)` crosses 0.9 V (half of VDD) on its 1st rising edge (`RISE=1`) and 2nd rising edge (`RISE=2`). The difference is exactly one period.

6.2 Results




```
period_val = 5.640303e-10  targ=8.961545e-10  trig=3.321242e-10
freq_val   = 1.772954e+09
```

Variable	Raw value	Human-readable	Meaning
`period_val`	5.640×10^{-10} s	564 ps	Time for one full oscillation cycle
`freq_val`	1.773×10^{9} Hz	**1.77 GHz**	Oscillations per second

> **Result:** At the typical process corner (TT), room temperature (27°C), and nominal 1.8 V supply, the 3-stage ring oscillator runs at **1.77 GHz**. This confirms that Sky130 130 nm transistors are capable of GHz-range operation.

---

7. PVT Analysis

7.1 What is PVT Analysis?

The TT simulation at 27°C represents ideal conditions. Real silicon faces manufacturing variations, voltage fluctuations, and temperature extremes. PVT analysis systematically tests each of these:

Letter	Stands for	What varies
P	Process	Manufacturing variation — transistors are faster (FF) or slower (SS)
V	Voltage	Supply voltage drops (low battery) or rises (overcharge)
T	Temperature	Chip heats under load or cools in a cold environment

7.2 Process Corners

The foundry cannot make every transistor identical. Instead of testing every possible variation, engineers test the extreme cases, called *corners*:

Corner	NMOS	PMOS	Models
`tt` — Typical-Typical	Normal	Normal	Most chips; ideal case
`ff` — Fast-Fast	Fast	Fast	Thin/short devices; max speed, high leakage
`ss` — Slow-Slow	Slow	Slow	Thick/long devices; min speed, low leakage
`fs` — Fast-Slow	Fast	Slow	Imbalanced; timing mismatch
`sf` — Slow-Fast	Slow	Fast	Imbalanced; opposite mismatch

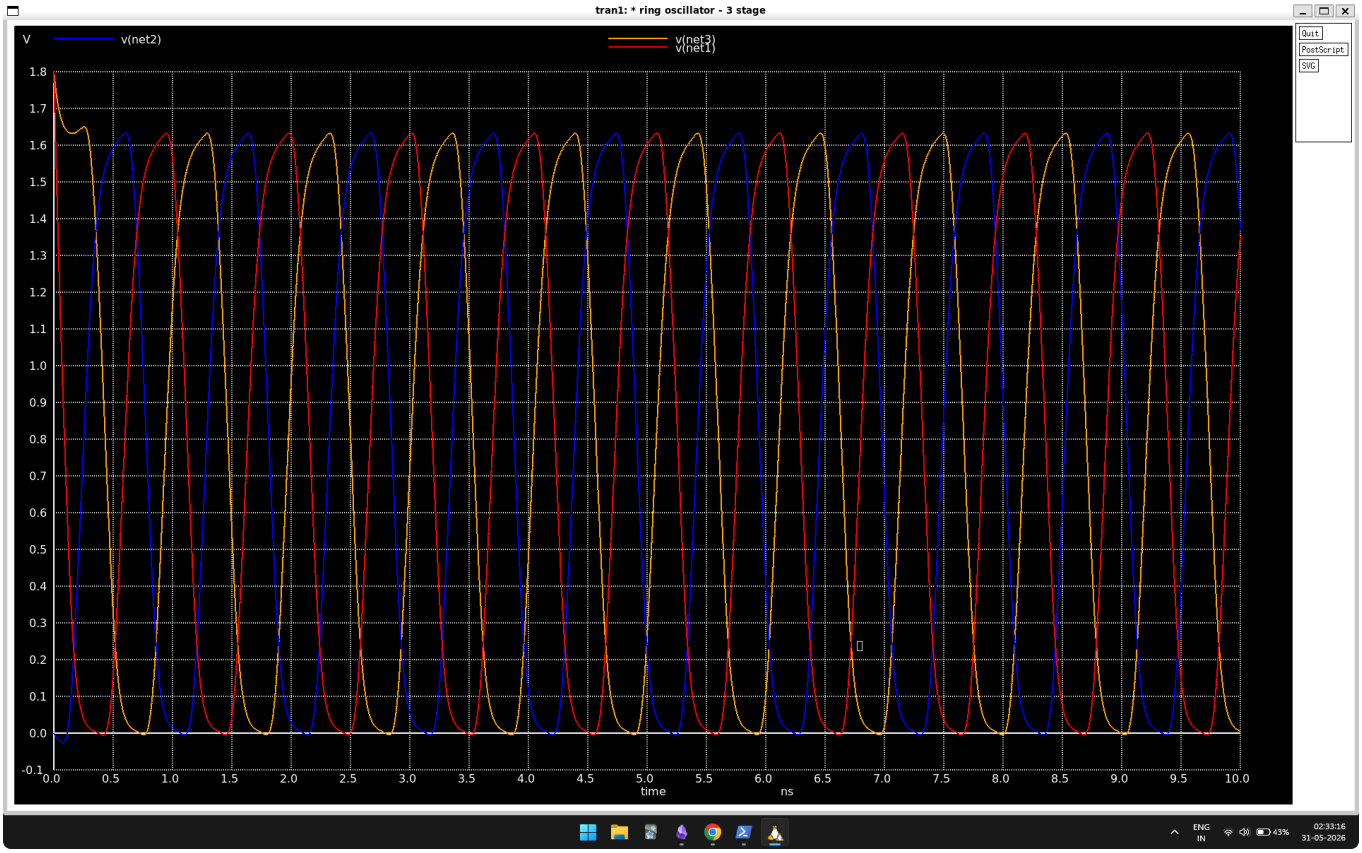
7.3 Worst-Case Corner: SS, 100°C, 1.62 V

This simulates the absolute worst case for chip performance: slow transistors, maximum operating temperature, and a 10% voltage droop (modelling IR drop on the power rail).

Only three lines in the netlist change:

```
* Change 1: Use Slow-Slow process corner
.lib /home/toney/pdk/sky130A/libs.tech/ngspice/sky130.lib.spice ss

* Change 2: Simulate at 100 degrees C (extreme operating temperature)
.option temp=100

* Change 3: Reduce supply by 10% (worst-case voltage droop)
VDD vdd 0 1.62
```

7.4 SS Corner Results and graph




```
Doing analysis at TEMP = 100.000000 and TNOM = 27.000000

Node                                   Voltage
----                                   -------
vdd                                       1.62
net1                                       1.8
vdd#branch                          4.1529e-05   <- leakage jumped from 0.32
nA to 41 µA!

period_val = 1.033766e-09
freq_val   = 9.673369e+08
```

7.5 Comparison of Results

Corner	Voltage	Temp	Frequency	vs. TT
TT (nominal)	1.8 V	27°C	**1.77 GHz**	Baseline
SS (worst case)	1.62 V	100°C	**967 MHz**	−45%

7.6 Analysis: Why the Frequency Dropped

Factor	Physical explanation
Lower voltage (1.62 V)	Reduces gate overdrive (V_GS − V_TH), which directly reduces drain current I_on. Since charging a capacitor takes time `t = C × ΔV / I`, less current means slower charging and a longer delay per stage.
High temperature (100°C)	Thermal lattice scattering increases — silicon atoms vibrate more aggressively, colliding with carriers and reducing electron and hole mobility.
SS process corner	Transistors have longer effective channel lengths and thicker gate oxides than designed, making them inherently slower to switch.
Leakage explosion	Subthreshold leakage increases exponentially with temperature. At 27°C the leakage was 3.2×10^{-10} A; at 100°C it jumped to 4.15×10^{-5} A — a 100,000× increase.

> **Industry significance:** This 45% frequency drop between TT and the worst-case SS corner is precisely why chip designers use Static Timing Analysis (STA). If a pipeline were clocked at 1.5 GHz assuming TT conditions, that chip would fail in the field once it heats up to 100°C. STA guarantees the design meets timing at every PVT corner before tape-out.

---

8. Key Learnings

1. **SPICE netlists.** Circuits are described as plain text: each line declares one component and its node connections. The component

letter prefix determines type (`V` = voltage source, `C` = capacitor, `X` = subcircuit).

2. **Ring oscillator principle.** An odd number of inverters in a feedback loop prevents any stable state, producing perpetual oscillation. Frequency is set by `f = 1 / (2 × N × t_pd)`.
3. **Sky130 subcircuit models.** Modern PDKs wrap transistors in subcircuits to include parasitics. This requires the `X` prefix in SPICE. Using `M` causes a model-not-found error.
4. **Process corners (PVT).** Manufacturing variations mean a chip can be fast-fast (FF) or slow-slow (SS). Engineers must verify the design works at every corner.
5. **Temperature and leakage.** High temperature causes lattice scattering (slower switching) and exponential increase in subthreshold leakage. The leakage at 100°C was 100,000× greater than at 27°C.
6. **STA motivation.** The 45% frequency drop from TT to SS worst-case is the quantitative reason Static Timing Analysis exists. A design that passes only at TT will fail in the field.
7. **Debugging SPICE.** Two key lessons: (a) never `.include` and `.lib` the same file — use only `.lib` to pick a specific corner; (b) volare requires an explicit commit hash when run outside an OpenLane project directory.

---

9. Commands Reference

9.1 Environment Setup

```
# --- PowerShell (Administrator) ---
wsl --install
wsl --install -d Ubuntu-22.04

# --- Ubuntu terminal ---
cd ~
sudo apt update && sudo apt upgrade -y
sudo apt install -y ngspice
sudo apt install -y python3 python3-pip
pip3 install volare
echo 'export PATH=$PATH:$HOME/.local/bin' >> ~/.bashrc
source ~/.bashrc
```

```
# Download Sky130 PDK
export PDK_ROOT=$HOME/pdk
volare enable --pdk sky130 --pdk-root $PDK_ROOT \
    bdc9412b3e468c102d01b7cf6337be06ec6e9c9a

echo 'export PDK_ROOT=$HOME/pdk' >> ~/.bashrc
source ~/.bashrc

# Verify
ls $PDK_ROOT/sky130A/libs.tech/ngspice/
```

9.2 Project Commands

```
mkdir -p ~/ic_projects/project1_ring_oscillator
cd ~/ic_projects/project1_ring_oscillator

# Create netlist
nano ring_oscillator.sp    # Ctrl+O then Enter to save, Ctrl+X to exit
# Tip: right-click to paste in WSL2 terminal

# Run simulation
ngspice ring_oscillator.sp

# Backup to Windows filesystem
cp -r ~/ic_projects /mnt/c/Users/Toney/Documents/ic_projects
```

9.3 nano Keyboard Reference

Shortcut	Action
`Ctrl + O`, then Enter	Save file
`Ctrl + X`	Exit (prompts to save if unsaved)
Right-click	Paste in WSL2 terminal
`Ctrl + K`	Cut current line
`Ctrl + U`	Paste cut line

---

10. Conclusion

This project successfully designed, simulated, and analysed a 3-stage CMOS ring oscillator using entirely open-source tools on a Windows 11 machine via WSL2/Ubuntu 22.04.

The ring oscillator was characterised across PVT conditions:

- At the nominal TT corner (27°C, 1.8 V): **1.77 GHz**
- At the worst-case SS corner (100°C, 1.62 V): **967 MHz**

The 45% frequency reduction between these two conditions directly demonstrates the importance of PVT-aware design and Static Timing Analysis in production silicon. A chip designed to run at 1.5 GHz assuming typical conditions would fail in the field under realistic thermal and voltage stress.

Next project: Full Custom Inverter Layout using Magic + KLayout + Sky130 — covering DRC (Design Rule Check) and LVS (Layout vs. Schematic), the physical design verification steps required before any circuit can be manufactured.

Project Completion Checklist

Task	Status
WSL2 + Ubuntu 22.04 setup	✅ Complete
ngspice 36 installed	✅ Complete
Sky130 PDK installed	✅ Complete
Ring oscillator netlist written	✅ Complete
Transient simulation (waveform)	✅ Complete
Frequency measurement (1.77 GHz)	✅ Complete
PVT analysis — SS worst case	✅ Complete
Layout	⬜ Deferred to Project 2

IC Design Series — Summer Projects · Project 1 of 8 · Tools: ngspice 36 + Sky130 PDK + WSL2 Ubuntu 22.04